Faeq Alrimawi
Computer Science & Information System
University of Limerick

Assignment 1: Parallel Numerical Integration

Due[1]: **Week 7 Thursday** (22/10/2026, 11:59pm Irish time)

Mark: **30%**

1 Objective

The objective is to explore the concepts of concurrency and parallelism by applying **Foster's methodology** and **Amdahl's Law** to parallelise a serial computational problem.

2 Submission Guidelines

Submission will be soft copy only on **Brightspace**.

- Two files: Report (pdf) and JAVA file (parallel code).

3 Documents to Submit

You should submit **TWO** files called **ID#_Report.pdf** and **ID#_Assignment1.java**. Replace the **ID#** with your student ID number. For example:

> if you are student number is 12345678, then you should name your files: **ID12345678_Report.pdf** & **ID12345678_Assignment1.java**

In your java file you should include prominent comments at the start of the code that contain your ID number and your Name. Brightspace will accept any filenames you use; however, you must adhere to the file naming requirements specified here.

NOTE: You will lose marks if you do not adhere to the submission requirements. Late submissions will be penalised.

[1] Early submission of drafts is advised. The Brightspace assignment will be set up to accept unlimited submissions, however, each one will overwrite the previous one. So each submission must be a single file.

4 Note on AI Usage for This Assignment

AI tools (e.g., ChatGPT, Copilot) can be used to support your work, not work instead of you. In particular:

You can use AI tools, for example, to:

- Brainstorm ideas or possible approaches.
- Refine or clarify your writing.
- Generate small code snippets (e.g., to a specific method).

However, AI tools cannot be used, for example, to:

- Generate the entire report, or most of its content.
- Generate the entire codebase or most of the code.
- Submit AI-generated text or code as your own work without appropriate checking, understanding, and adaptation.

5 Note on Plagiarism and Cheating

Plagiarism is a **serious academic offense** that undermines the integrity of the learning process and the reputation of both individuals and institutions. In your work, please be considerate of the following:

- **Original Work:** All work submitted must be original and created by you. Copying, reusing, or paraphrasing content from external sources without proper attribution is strictly prohibited.
- **Proper Citation:** If you refer to external sources, including textbooks, online materials, or classmates' work, ensure that you provide appropriate citations. Failure to do so can be considered plagiarism.
- **Collaboration vs. Copying:** Collaboration between students is encouraged and expected, but there is a clear distinction between collaborative work and copying. Collaboration involves sharing ideas and discussing concepts to enhance understanding. Copying involves duplicating another person's work or ideas without adding your own input.

Consequences for plagiarism and cheating may include a failing grade for the assignment, academic probation, or more severe disciplinary actions as per university policies.

More information on plagiarism, academic integrity, and academic misconduct can be found here: https://www.ul.ie/provost/functions-processes/academic-integrity-unit/academic-integrity-and-academic-misconduct

6 Assignment Specification: What You Need to Do

You are tasked to convert a **numerical integration** problem from a serial to a parallel solution using **Foster's methodology**. You need to implement both versions in Java (the serial implementation will be provided), apply **Amdahl's Law** to analyse performance, and compare the actual speedup achieved on your machine. The assignment also requires you to write a report discussing the methodology, implementation, and performance analysis, along with the machine specifications used for testing.

6.1 Background: Numerical Integration Using the Trapezoidal Rule

Numerical integration is the process of approximating the value of an integral when the exact solution cannot be easily computed analytically. It is a fundamental concept in computational mathematics, and is widely used in various fields such as physics, engineering, and computer science, where finding the area under a curve is necessary. In this assignment, you will focus on solving a **definite integral** numerically using the **Trapezoidal Rule**.

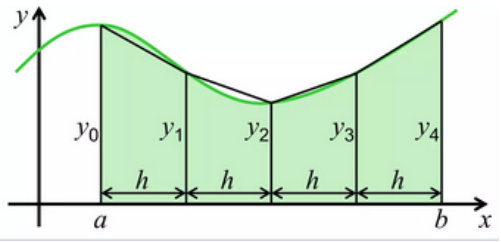
The Trapezoidal Rule approximates the area under a curve by dividing it into small trapezoids rather than rectangles. The more trapezoids the area is divided into the more accurate the estimate.

Formula for the Trapezoidal Rule:

$$\int_a^b f(x)dx \approx \tfrac{1}{2}h(y_0 + 2y_1 + 2y_2 + \ldots + 2y_{n-1} + y_n)$$

Where:

- a is the lower limit of the integral.
- b is the upper limit of the integral.
- n is the number of intervals or trapezoids.
- h = (b−a)/n is the width of each interval.
- $y_n = f(a+nh)$



For example, the integration of *f*(x) = x over the period from 1 to 2 is: $\int_1^2 x\,dx = 1.5$. Using the Trapezoidal Rule, this can be calculated as follows assuming we want to use 4 trapezoids.

$$h = \frac{2-1}{4} = 0.25$$

$$\int_1^2 x\,dx = \frac{1}{2} \times 0.25 \times \big(f(1) + 2 \times f(1.25) + 2 \times f(1.5) + 2 \times f(1.75) + f(2)\big)$$

$$\int_1^2 x\,dx = \frac{1}{2} \times 0.25 \times (1 + 2.5 + 3 + 3.5 + 2)$$

$$\int_1^2 x\,dx = 1.5$$

6.2 Problem Description: Serial Algorithm

You will be working on a numerical integration problem using the Trapezoidal Rule. You are provided with a serial Java implementation of this algorithm, which computes the integral of a function over a given interval by dividing the area under the curve into trapezoids and summing their areas.

While the serial solution works correctly, it processes each trapezoid sequentially, which limits its performance, especially when computing with a large number of intervals. To make the computation more efficient and take advantage of modern multi-core processors, we want to parallelise the solution.

Serial Code is available as an attachment to the assignment on Brightspace.

6.3 Parallelise Using Foster's Methodology (40%)

Your first task is to apply **Foster's Methodology** to break down the serial code and create a parallel implementation that can utilise multiple threads to perform the integration faster.

Provide a detailed explanation of how you applied each of the steps to the problem. Include diagrams or pseudocode if necessary.

6.4 Implement Your Parallel Solution (20%)

Based on your design from Foster's methodology, implement the parallel version of the numerical integration problem in Java. You can use Java's concurrency features such as Thread, and ForkJoinPool.

6.5 Performance Analysis (40%)

You are required to measure the execution time of both the serial and parallel implementations. Then apply Amdahl's Law to calculate the speedup achieved through parallelism and analyse how the performance improves with multiple threads.

6.5.1 Execution Time Measurement

Measure the execution time of both the serial and parallel versions of your program for different numbers of intervals.

- `n` values: 10 million, 100 million, and 500 million
- Thread counts: 2, 4, 8, 16.

6.5.2 Speedup Calculation

For each combination of `n` and thread count in the previous section, compute the **actual speedup**.

Using Amdahl's law, compute the **theoretical speedup** of each of the combinations.

You are also required to calculate the **maximum theoretical speedup** using Amdahl's law.

Include a section in your report detailing your machine's specifications (e.g., CPU model, number of cores) and discuss how these might impact the performance of the parallel implementation.

7 Detailed Report

Submit a detailed report covering the following:

- Foster's Methodology: Explain how you applied Foster's methodology to parallelise the problem.
- Parallel Implementation: Discuss your Java implementation, including design decisions, thread management, and synchronisation mechanisms, if any.
- Performance Analysis: Include speedup calculations, theoretical analysis using Amdahl's Law, and performance graphs for different thread counts and problem sizes.

8 Assessment Criteria

The table shown below outlines the assessment criteria and grading breakdown for evaluating your performance. Utilise these guidelines as a reference and consistently strive for excellence in your work.

Table 1 Assessment Criteria

Criteria	Excellent (80%-100%)	Proficient (60%-79%)	Satisfactory (40%-59%)	Limited (20%-39%)	Inadequate (0%-19%)
Foster Methodology (40%)					
Parallel Implementation (20%)					
Performance Measurement (40%)					

Final Grade: (<Mark>/100)*30%

Table 2 Scale Description

Scale	Description
Excellent	Exceptional understanding and execution; consistently exceeds objectives; comprehensive, highly detailed, with minimal or no errors.
Proficient	Strong understanding and execution; consistently meets or slightly exceeds objectives; detailed with good depth and few errors.
Satisfactory	Meets basic expectations; adequate quality and coverage but may lack depth or precision and contain occasional errors.
Limited	Some understanding and effectiveness, but inconsistent execution; gaps in depth, precision, or coverage.
Inadequate	Does not meet minimum expectations; significant deficiencies, poor or incomplete execution, with multiple errors and omissions.